

Introduction

When it comes to query languages, SQL is the most widely recognized standard. The majority of developers have experience and are comfortable with SQL. For this reason Orient DB uses SQL as its query language and adds some extensions to enable graph functionality. There are a few differences between the standard SQL syntax and that supported by OrientDB, but for the most part, it should feel very natural. The differences are covered in the [OrientDB SQL dialect](#) section of this page.

If you are looking for the most efficient way to traverse a graph, we suggest to use the [SQL-Match](#) instead.

Many SQL commands share the [WHERE condition](#). Keywords and class names in OrientDB SQL are case insensitive. Field names and values are case sensitive. In the following examples keywords are in uppercase but this is not strictly required.

If you are not yet familiar with SQL, we suggest you to get the course on [KhanAcademy](#).

For example, if you have a class `MyClass` with a field named `id`, then the following SQL statements are equivalent:

```
SELECT FROM MyClass WHERE id = 1
select from myclass where id = 1
```

The following is NOT equivalent. Notice that the field name 'ID' is not the same as 'id'.

```
SELECT FROM MyClass WHERE ID = 1
```

Automatic usage of indexes

OrientDB allows you to execute queries against any field, indexed or not-indexed. The SQL engine automatically recognizes if any indexes can be used to speed up execution. You can also query any indexes directly by using `INDEX:<index-name>` as a target. Example:

```
SELECT FROM INDEX:myIndex WHERE key = 'Jay'
```

Extra resources

- [SQL syntax](#)
- [SQL projections](#)
- [SQL conditions](#)
- [Where clause](#)
- [Operators](#)
- [Functions](#)
- [Pagination](#)
- [Pivoting-With-Query](#)
- [SQL batch](#)
- [SQL-Match](#) for traversing graphs

OrientDB SQL dialect

OrientDB supports SQL as a query language with some differences compared with SQL. Orient Technologies decided to avoid creating Yet-Another-Query-Language. Instead we started from familiar SQL with extensions to work with graphs. We prefer to focus on standards.

If you want learn SQL, there are many online courses such as:

- [Online course Introduction to Databases by Jennifer Widom from Stanford university](#)
- [Introduction to SQL at W3 Schools](#)
- [Beginner guide to SQL](#)
- [SQLCourse.com](#)
- [YouTube channel Basic SQL Training by Joey Blue](#)

To know more, look to [OrientDB SQL Syntax](#).

Or order any book like [these](#)

No JOINS

The most important difference between OrientDB and a Relational Database is that relationships are represented by **LINKS** instead of JOINS.

For this reason, the classic JOIN syntax is not supported. OrientDB uses the “dot (.) notation” to navigate **LINKS**. Example 1 : In SQL you might create a join such as:

```
SELECT *  
FROM Employee A, City B  
WHERE A.city = B.id  
AND B.name = 'Rome'
```

In OrientDB, an equivalent operation would be:

```
SELECT * FROM Employee WHERE city.name = 'Rome'
```

This is much more straight forward and powerful! If you use multiple JOINS, the OrientDB SQL equivalent will be an even larger benefit. Example 2: In SQL you might create a join such as:

```
SELECT *  
FROM Employee A, City B, Country C,  
WHERE A.city = B.id  
AND B.country = C.id  
AND C.name = 'Italy'
```

In OrientDB, an equivalent operation would be:

```
SELECT * FROM Employee WHERE city.country.name = 'Italy'
```

Projections

In SQL, projections are mandatory and you can use the star character `*` to include all of the fields. With OrientDB this type of projection is optional. Example: In SQL to select all of the columns of Customer you would write:

```
SELECT * FROM Customer
```

In OrientDB, the `*` is optional:

```
SELECT FROM Customer
```

See [SQL projections](#)

DISTINCT

In OrientDB v 3.0 you can use DISTINCT keyword exactly as in a relational database:

```
SELECT DISTINCT name FROM City
```

Until v 2.2, DISTINCT keyword was not allowed; there was a DISTINCT() function instead, with limited capabilities

```
//legacy
```

```
SELECT DISTINCT(name) FROM City
```

HAVING

OrientDB does not support the **HAVING** keyword, but with a nested query it's easy to obtain the same result. Example in SQL:

```
SELECT city, sum(salary) AS salary
FROM Employee
GROUP BY city
HAVING salary > 1000
```

This groups all of the salaries by city and extracts the result of aggregates with the total salary greater than 1,000 dollars. In OrientDB the **HAVING** conditions go in a select statement in the predicate:

```
SELECT FROM ( SELECT city, SUM(salary) AS salary FROM Employee GROUP BY city
) WHERE salary > 1000
```

Select from multiple targets

OrientDB allows only one class (classes are equivalent to tables in this discussion) as opposed to SQL, which allows for many tables as the target. If you want to select from 2 classes, you have to execute 2 sub queries and join them with the **UNIONALL** function:

```
SELECT FROM E, V
```

In OrientDB, you can accomplish this with a few variable definitions and by using the **expand** function to the union:

```
SELECT EXPAND( $c ) LET $a = ( SELECT FROM E ), $b = ( SELECT FROM V ), $c =
UNIONALL( $a, $b )
```